

COMP 3311

DATABASE MANAGEMENT SYSTEMS

LECTURE 22

TRANSACTION MANAGEMENT: TRANSACTIONS

TRANSACTION MANAGEMENT: OUTLINE

Transactions

Concurrency Control Protocols

Recovery System

NoSQL Transactions

OVERVIEW

A *transaction* is a *unit of program execution* that accesses, and possibly updates, the database.

- A transaction must preserve database consistency.
 - During transaction execution the database may be inconsistent.
 - However, when the transaction commits (ends), the database must be consistent.
- Two main issues to deal with:
 - **Concurrent execution** of multiple transactions.
 - Handled by concurrency control.
 - **Failures** of various kinds (e.g., hardware, system crashes, etc.).
 - Handled by database recovery.

ACID PROPERTIES

Atomicity

Either all instructions of a transaction are properly **reflected** in the database, or none are.

Consistency

Execution of a transaction *in isolation* (i.e., it is the only transaction executing) **preserves the consistency** of the database.

Isolation

Concurrently executing transactions **must be unaware** of other concurrently executing transactions. **Intermediate results** of a transaction **must be hidden** from other concurrently executing transactions.

- For every pair of transactions T_i and T_j , it appears to T_i that either T_j **finished execution before** T_i started or that T_j **started execution after** T_i finished (i.e., either T_i executes before T_j or T_j executes before $T_i \Rightarrow$ **serial execution**).

Durability

After a transaction completes successfully, the **changes** it made to the database **persist**, **even if there are system failures**.

ACID PROPERTIES: EXAMPLE

Consider a transaction to transfer \$50 from account A to account B.

Atomicity Requirement

Handled by recovery system.

If the transaction fails *after step 3* and *before step 6*, the system should ensure that any of the transaction's updates (i.e., `write(A)`) are not reflected in the database; else an inconsistency will result (i.e., the sum $A + B$ will not be preserved).

transaction failure

1. `read(A)`
2. $A := A - 50$
3. `write(A)`
4. `read(B)`
5. $B := B + 50$
6. `write(B)`

Consistency Requirement

Handled by concurrency control.

The sum of A and B is unchanged by the execution of the transaction.

ACID PROPERTIES: **EXAMPLE** (CONTD)

Isolation Requirement Handled by concurrency control.

If *between steps 3 and 6*, another transaction can access the partially updated database, it will see an inconsistent database (i.e., the sum $A + B$ will be less than it should be). Isolation can be ensured trivially by executing transactions *serially* (i.e., one at a time, one after the other). However, *executing multiple transactions concurrently* has significant benefits.

transaction failure

1. read(A)
2. $A := A - 50$
3. write(A)
4. read(B)
5. $B := B + 50$
6. write(B)

Durability Requirement Handled by concurrency control & recovery system.

Once the user has been notified that the transaction has completed (i.e., the transfer of the \$50 has taken place), the *updates to the database by the transaction must persist despite failures*.

TRANSACTION STATE

Active

The initial state; the transaction stays in this state while executing.

Partially committed

After the final instruction in the transaction has been executed but *before it finally commits*.

Failed

After the discovery that normal execution can no longer proceed.

Aborted

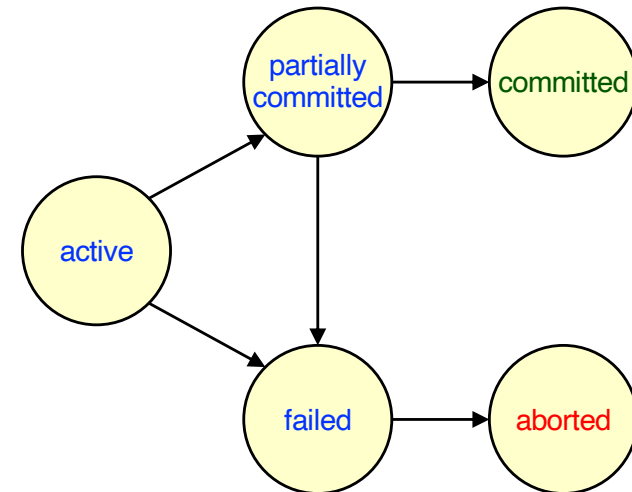
After the transaction has been **rolled back** (i.e., its effects are undone) and the **database restored to its state prior to the start of the transaction**.

There are two options for how to handle a transaction after it has been aborted:

- **restart** the transaction – only if there is no internal logic error.
- **kill** the transaction – if there is an internal logic error.

Committed

After *successful completion*.



CONCURRENT EXECUTIONS AND ISOLATION

- Advantages of **concurrent transaction execution**:
 - **increased system utilization**, leading to **better transaction throughput** since one transaction can be using the CPU while another is reading from or writing to secondary storage.
 - **reduced average response time** for transactions. Short duration transactions do not need to wait behind long duration ones.

✎ **Concurrent execution, if not handled properly, can lead to violation of the isolation property.**

(A transaction can see the partial, incomplete results of another transaction).

- **Concurrency control schemes** are mechanisms used to **achieve isolation** in the presence of **concurrent executions**.
 - ✎ They **control the interaction** among the concurrent transactions **to prevent them from destroying the database consistency** due to problems that can arise when concurrent transactions **access the same data**.

SCHEDULES

- A **schedule** is a **sequence of instructions** showing the **chronological order** in which instructions of **concurrent transactions** are executed.
 - A schedule for a set of transactions **must** consist of all instructions of those transactions.
 - A schedule **must** preserve the order in which the instructions appear in each individual transaction.
- To **preserve database consistency**, a schedule for executing transactions must be one of the following.
 - serial** Transactions are executed **one at a time**, **one after another**.
⇒ All the instructions of a transaction are executed **before** the next transaction starts executing.
 - serializable** Transactions are executed concurrently (i.e., at the same time, all together) in an **interleaved** manner so that their execution is **equivalent** to a serial schedule.

SERIAL SCHEDULE

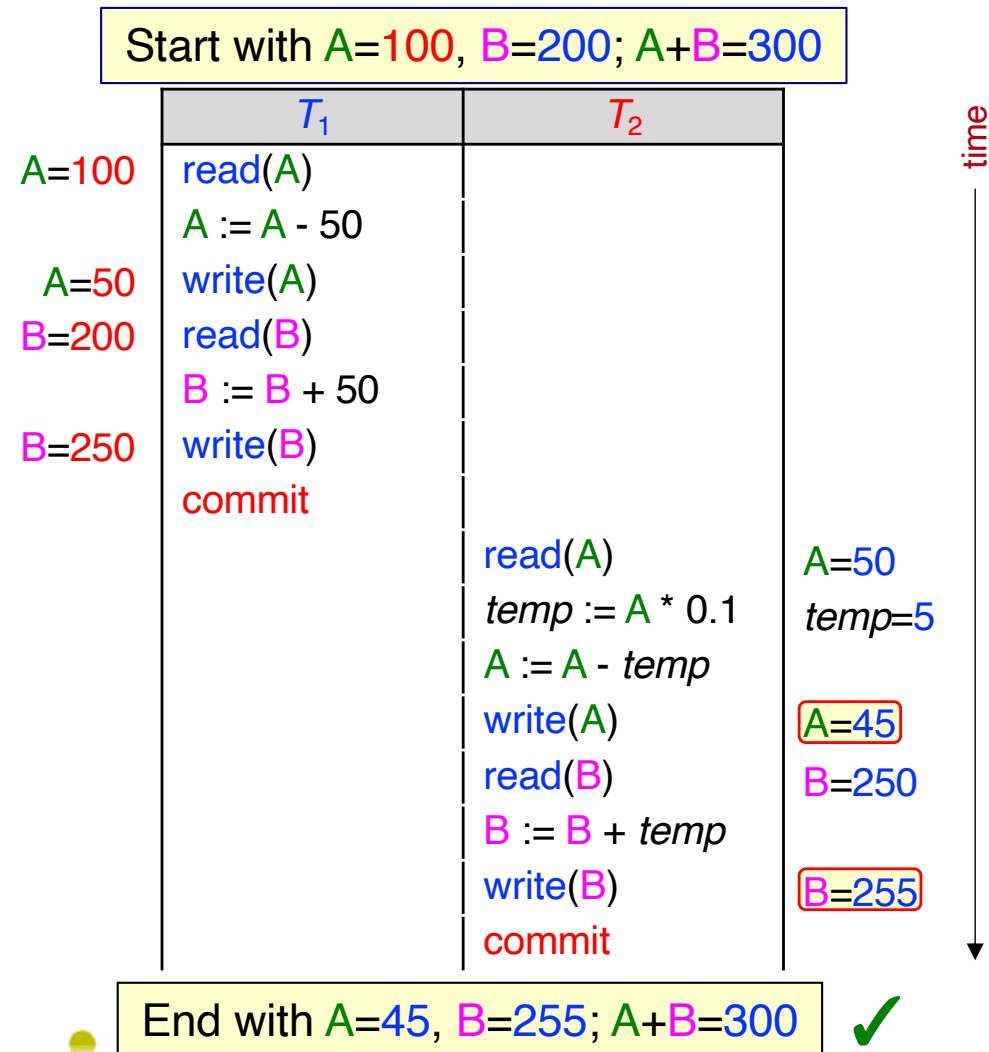
T_1 transfers \$50 from A to B; T_2 transfers 10% of the balance from A to B.

A **serial schedule** is shown to the right, in which T_1 is executed first followed by T_2 .

The consistency requirement is that the sum of A + B is the same after the transactions execute as it was before they executed.

Note: A schedule in which T_2 is followed by T_1 is also a serial schedule (*but may have a different result*).

👉 For n transactions there exist $n!$ different valid serial schedules.

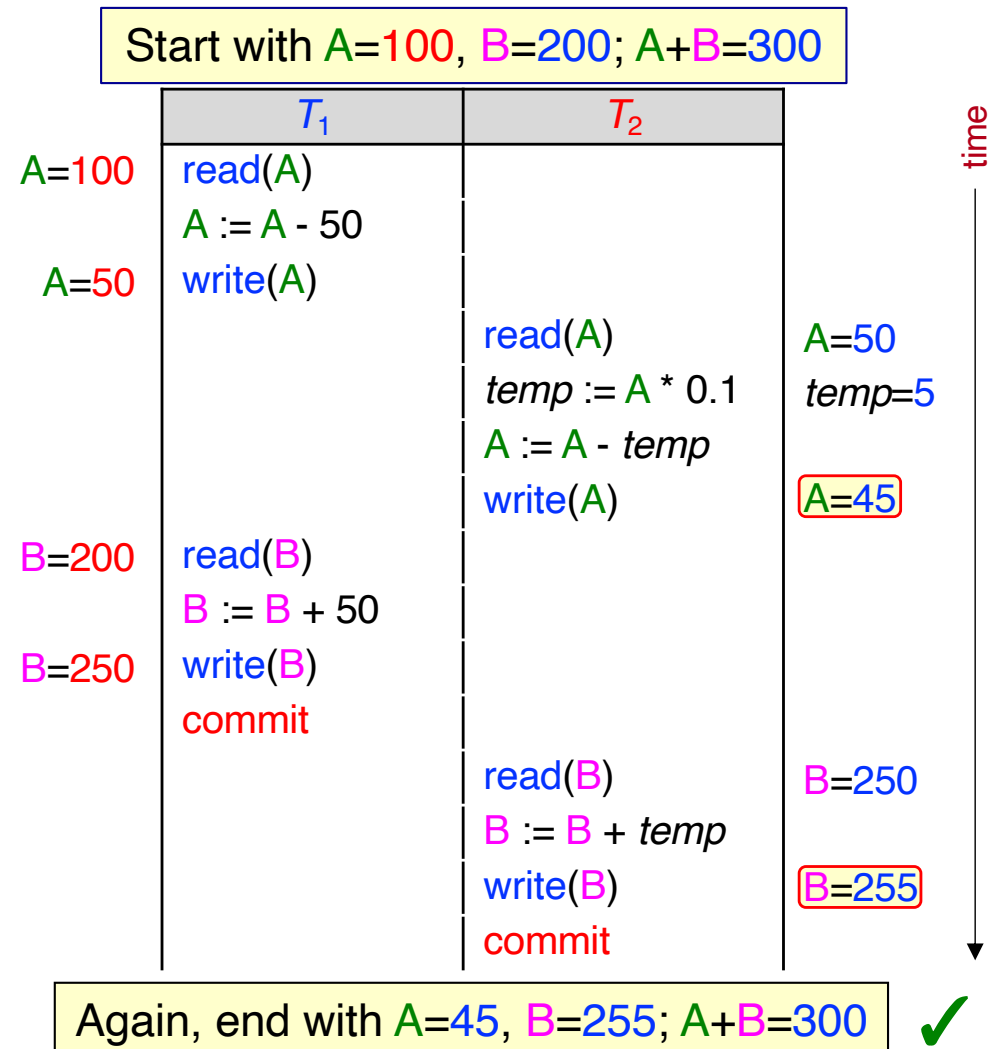


NOT SERIAL BUT CORRECT SCHEDULE

T_1 transfers \$50 from A to B; T_2 transfers 10% of the balance from A to B.

✎ The value of sum **A + B** is the same after the schedule executes as it was before it executed (i.e., consistency is preserved).

This schedule is equivalent to the previous serial schedule since consistency is preserved.

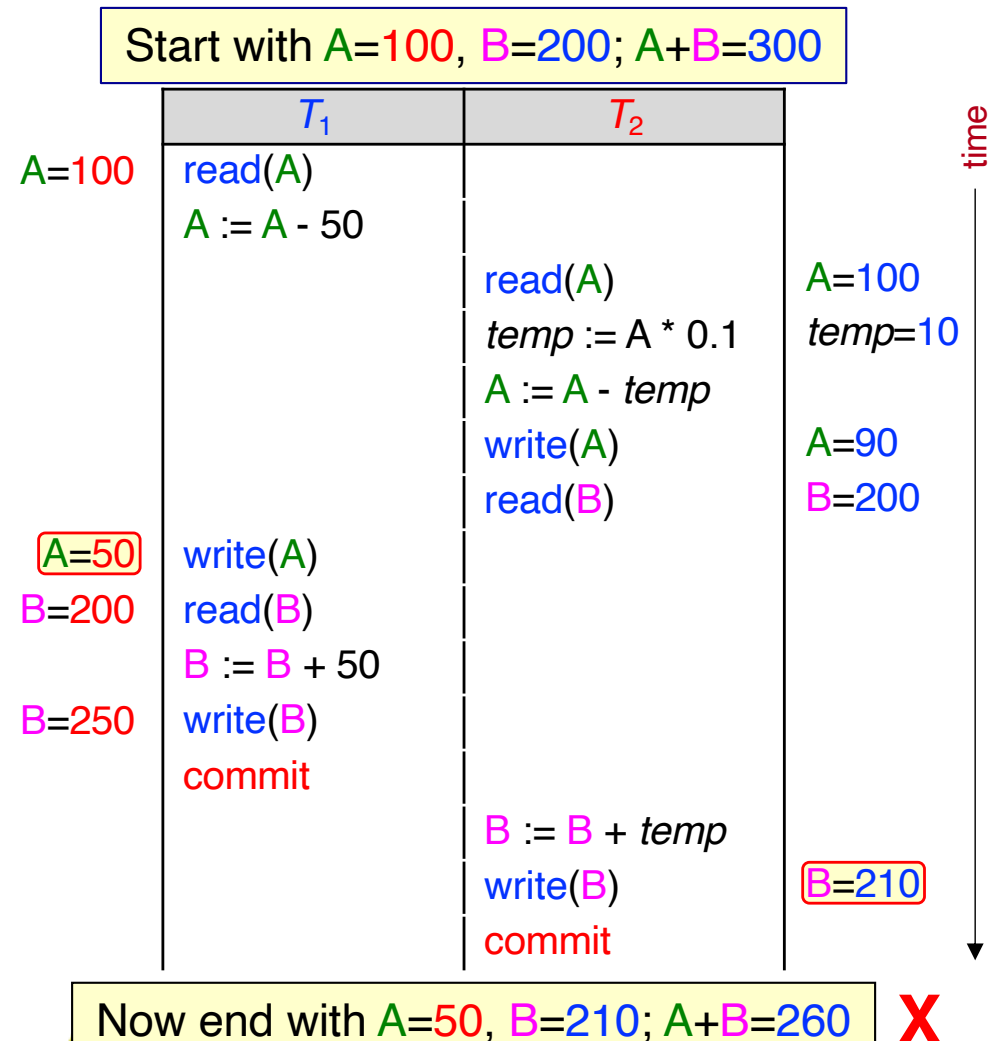


NOT SERIAL AND INCORRECT SCHEDULE

T_1 transfers \$50 from A to B; T_2 transfers 10% of the balance from A to B.

✋ The value of sum $A + B$ is the not same after the schedule executes as it was before it executed (i.e., consistency is not preserved).

This schedule is incorrect and should not be allowed since consistency is not preserved.



SERIALIZABILITY

Basic Assumption

Each transaction preserves database consistency.

✎ A serial execution of a set of transactions always preserves database consistency.

- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule.
- We ignore instructions other than read and write instructions and assume that transactions may perform arbitrary computations on data in the buffer in between reads and writes.

✎ Our simplified schedules consist of only read and write instructions.

CONFLICT SERIALIZABILITY

Instruction I_i of transaction T_i and instruction I_j of transaction T_j **conflict if and only if** there exists some data item Q accessed by both I_i and I_j and **at least one of these instructions is write(Q).**

- | | |
|--|---|
| 1. $I_i = \text{read}(Q), I_j = \text{read}(Q).$ | I_i and I_j <u>do not</u> conflict. |
| 2. $I_i = \text{read}(Q), I_j = \text{write}(Q).$ | I_i and I_j conflict . |
| 3. $I_i = \text{write}(Q), I_j = \text{read}(Q).$ | I_i and I_j conflict . |
| 4. $I_i = \text{write}(Q), I_j = \text{write}(Q).$ | I_i and I_j conflict . |

- Intuitively, a conflict between two instructions I_i and I_j of two different transactions T_i and T_j forces a (logical) temporal order between them.
- If I_i and I_j are consecutive in a schedule and they **do not conflict**, the result of their execution would remain the same even if they had been **interchanged in the schedule**.

CONFLICT SERIALIZABILITY (CONTD)

Conflict Equivalent

Schedules S and S' are **conflict equivalent** if S can be **transformed** into S' by a series of **swaps** (interchanges) of **non-conflicting** instructions.

Conflict Serializable

A schedule S is **conflict serializable** if it is **conflict equivalent** to a serial schedule.

Example: A schedule that is not conflict serializable.

Diagram illustrating three transaction schedules and their equivalence to serial schedules:

- Left Schedule:** A parallel execution of two transactions T_1 and T_2 .

T_1	T_2
read(Q)	
	write(Q)
write(Q)	
- Middle Schedule:** A serial execution of T_1 followed by T_2 , labeled "serial schedule T_1T_2 ".

T_1	T_2
read(Q)	
write(Q)	
	write(Q)
- Right Schedule:** A serial execution of T_2 followed by T_1 , labeled "serial schedule T_2T_1 ".

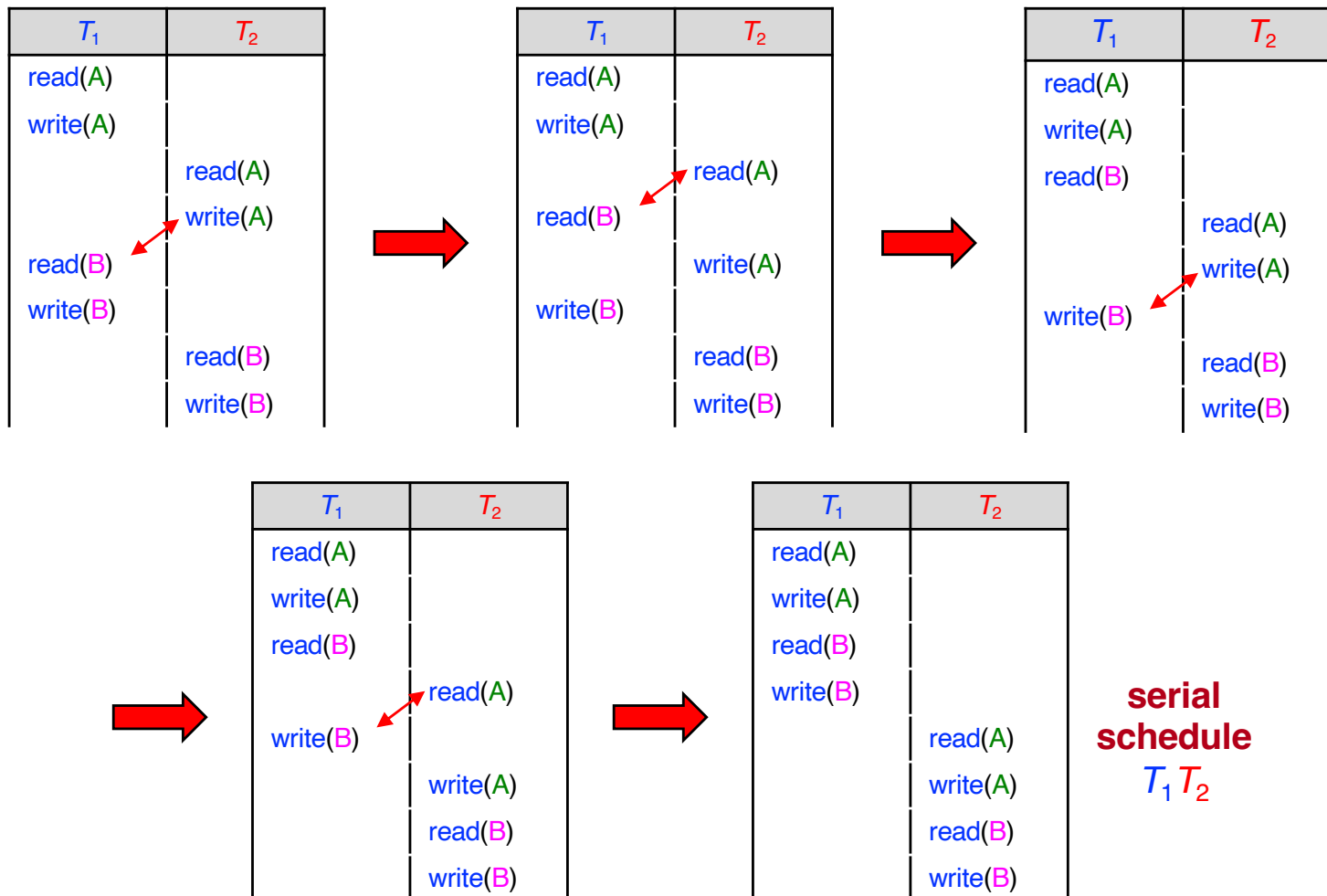
T_1	T_2
	write(Q)
read(Q)	
write(Q)	

Red $\neq$ symbols indicate that the left schedule is not equivalent to either of the serial schedules.

👉 The instructions in the schedule cannot be swapped to obtain either the serial schedule T_1T_2 or the serial schedule T_2T_1 because the $\text{write}(Q)$ instructions conflict (i.e., the final value of Q is different).

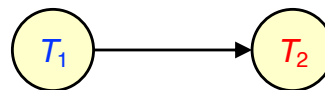
CONFLICT SERIALIZABLE SCHEDULE

This schedule can be shown to be equivalent to the serial schedule T_1T_2 by performing a series of swaps (interchange) of non-conflicting instructions.

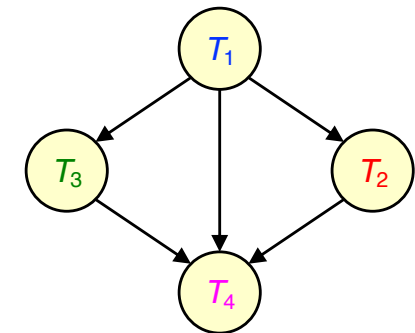
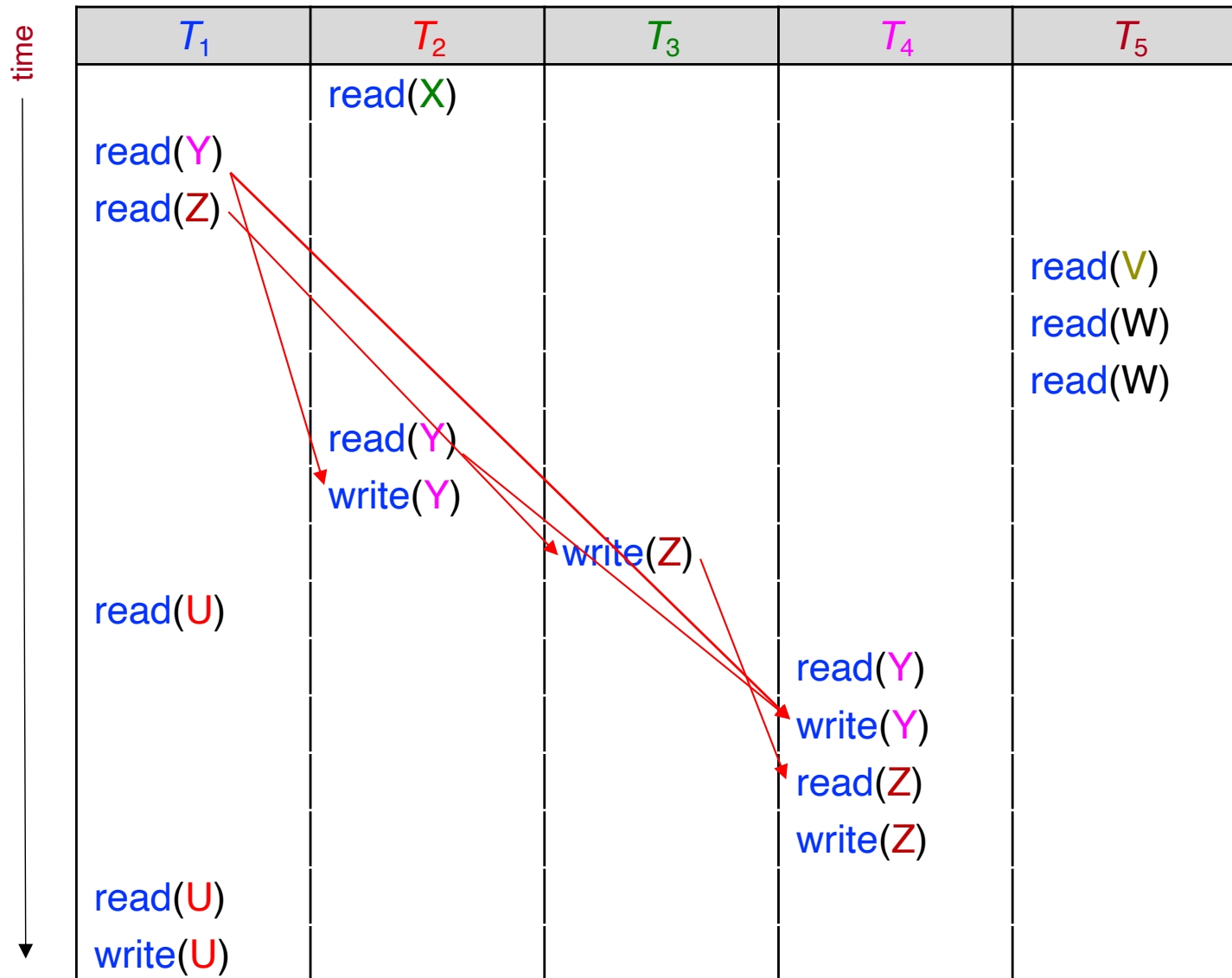


TESTING FOR SERIALIZABILITY

- Consider a schedule consisting of a set of transactions $T_1 T_2 \dots T_n$.
- A **precedence graph** is a **directed graph** where the vertices are the transactions (i.e., the transaction names).
- We draw an arc from T_i to T_j if the two transactions conflict and T_i **accessed the data item** on which the conflict arose earlier in the schedule than T_j .
- We can label the arc by the data item that was accessed, if desired (but we usually do not do so).



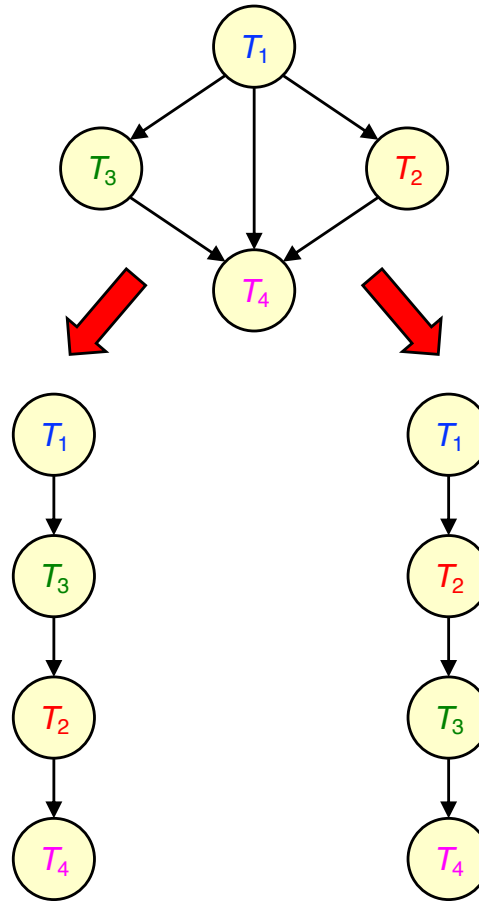
EXAMPLE SCHEDULE & PRECEDENCE GRAPH



Precedence
graph for the
schedule.

CONFLICT SERIALIZABLE PRECEDENCE GRAPH

If the precedence graph can be **topologically sorted** to find a **linear order** consistent with the partial order of the precedence graph, then the schedule is serializable.



T_1 must execute before T_2 , T_3 and T_4 .
 T_2 and T_3 must execute before T_4 .
 T_2 and T_3 can execute in any order.

✋ If a precedence graph has a **cycle**, then the schedule is **not** conflict serializable.

Why?

RECOVERABILITY

A schedule is **recoverable** if a transaction T_j that reads a data item *previously written* by another transaction T_i commits after T_i .

☞ If T_i aborts, then T_j should not have read the data item written by T_i . Thus, T_j should also be aborted. However, if T_j has already committed, then it cannot be aborted.

☞ **A schedule that is not recoverable violates the durability property.**

T_2 reads a data item, A , written by T_1 .

T_2 commits immediately after the read.

If T_1 aborts, which will undo the write of A , then T_2 should also abort since it read data item A written by T_1 .

However, since T_2 has committed, it cannot abort and so can show to the user an inconsistent database state **violating the durability property**.

Example

T_1	T_2
read(A)	
write(A)	
	read(A)
	commit
read(B)	

☞ **All schedules must be recoverable.**

CASCADING ROLLBACK

- **Cascading rollback** happens when a single transaction rollback (failure) leads to a series of transaction rollbacks (failures).
- This schedule is recoverable, because every transaction T_i commits after all transactions that wrote data items which T_i read. However, if T_1 fails, T_2 and T_3 must also be rolled back.
- ✎ **A transaction commit is a partial commit.**
- This can lead to the undoing of a significant amount of work.

How would you (re)order the commit statements to make the schedule cascadeless?

T_1	T_2	T_3
read(A)		
write(A)		
	read(A)	
	write(A)	
		read(A)
commit		
	commit	
		commit

For a schedule to be cascadeless, a transaction should only read a data item after the transaction that wrote it commits; else an abort of the transaction that wrote the data item can cause an abort of the transaction that read the data item.

CASCADELESS SCHEDULES

A schedule is **cascadeless** if, for each pair of transactions T_i and T_j where T_j reads a data item previously written by T_i , the **commit** instruction of T_i appears before the **read** instruction of T_j .
 T_j does not read a data item written by T_i until after T_i commits.

- Cascadeless schedules are schedules where cascading rollbacks cannot occur.

 **Every cascadeless schedule is also recoverable.**

- It is **highly desirable** to restrict the schedules to those that are cascadeless to avoid rollbacks of other transactions when a transaction aborts.

IMPLEMENTATION OF ISOLATION

- Schedules **must be serializable**, and **recoverable**, to ensure database consistency, and **preferably, cascadeless**.
 - A policy whereby only one transaction can execute at a time generates serial schedules but provides no concurrency.

 **Testing a schedule for serializability after it has executed is too late!**

Goal: Develop **protocols** that **both allow concurrency and ensure serializability**.

- A protocol will generally **not** examine the precedence graph as it is being created; instead, it will **impose a discipline on the order in which data items are accessed**, which all transactions must follow, that **avoids nonserializable schedules** (i.e., it will restrict the allowable schedules).

 **Concurrency control protocols trade-off between the amount of concurrency allowed and the amount of overhead that is incurred to ensure that schedules are serializable and recoverable.**